

Decoupled VPS and Local Architecture

https://gemini.google.com/app/c1e6fea39dea0d22

User prompt: I will buy domain name from cloudflare, to connect it with VPS aws lightsail. Must make sure it will be Stealth and use any fake page for opening. Connect the main branch only .. While keep the dev branch local. Must make sure Binance API is only connected to the live vps ... Sql only for live vps .. local is completely discounted from the vps .. even the telegram bot is separated Everything should be separated even the pm2 ..

Response: Decoupled Stealth Architecture Overview

To achieve total operational stealth, zero-capital leakage, and absolute separation between development and execution, the system divides into two completely isolated environments: a **Production Lightsail VPS** running live operations, and a **Local Development Sandbox** for coding and offline research.

Production vs. Local Isolation Matrix

Architectural Layer	Live AWS Lightsail VPS (Production)	Local Workstation (Development)
Git Branch Tracking	Strictly main branch	Strictly dev / feature branches
Binance API Connection	Live API Keys (Static IPv4 Whitelisted, Withdrawals Disabled)	Disconnected / Testnet / Offline Simulation Mode
Database & Persistence	VPS Local PostgreSQL (Sub-millisecond loopback, zero quotas)	Disconnected / Offline Mock JSON buffers
Telegram Bot Dispatcher	Live Trade Alerts & Interactive 2-Way Command Center	Disabled (TELEGRAM_ENABLED=false) or dedicated sandbox bot
Process Manager (PM2)	Live headless execution daemon + Next.js standalone process	Local dev server (npm run dev) or local mock daemon
Domain & Network Edge	Cloudflare Proxied (Orange Cloud) + Caddy/Nginx Reverse Proxy	Localhost (127.0.0.1) only

Stealth & Front-Facing Decoy Strategy

- Cloudflare DNS Proxying:** Enable Cloudflare Proxy (*Orange Cloud*) for your domain DNS records. This completely masks your AWS Lightsail dedicated static IPv4 address from public port scanners and indexers.
- Decoy Root Landing Page (/):** Configure the root domain to display a lightweight, non-financial decoy page (such as a generic developer telemetry health monitor, an empty static holding page, or a raw { "status": "healthy", "service": "telemetry-worker" } JSON response).
- Private Cockpit Routing:** Host your actual trading terminal cockpit and M2M/MCP gateways behind an obfuscated subdomain (e.g., core.yourdomain.xyz) secured strictly by Google OAuth or Cloudflare Zero Trust Access.
- Default Fail-Closed Gateways:** Ensure all unauthenticated hits to protected routes immediately resolve to a standard 404 Not Found or 403 Forbidden rather than exposing login interfaces or financial schemas.

Strict Decoupling Rules

- Environment Variable Containment:** Never commit .env files to Git. The production .env lives exclusively on the Lightsail VPS filesystem, while your local machine uses .env.local configured with mock credentials and offline fallbacks.
- Fail-Safe Live Gate in Codebase:** Ensure the quantitative execution loop contains an explicit environment guard: live order routing is blocked unless the runtime environment explicitly validates production flags and whitelisted server identifiers.
- Telegram State Deduplication:** The live VPS maintains its own in-memory deduplication set and local ledger, preventing notification crossover or duplicate order event spamming.

Master IDE Directive: Environment Isolation & Stealth Audit

Copy and paste this directive into your IDE to ensure your application enforces strict separation between the local development environment and the live production VPS:

Role & Objective:

You are an elite quantitative architect and Next.js systems engineer. We are preparing our trading terminal for decoupled deployment: the live AWS Lightsail VPS will track 1

Architectural Directives:

- Environment & API Guardrails:
 - Audit all exchange connectors, execution runners, and data fetchers.
 - Enforce a strict environment check: live Binance Futures REST/WebSocket order routing must only initialize when explicitly configured with production environment flags
 - Ensure local development defaults safely to Offline Simulation Mode or Binance Testnet when live exchange keys are omitted, with zero runtime exceptions.
- Database Decoupling & Local Fallbacks:
 - Ensure the database connection pool gracefully handles disconnected or local offline states without blocking chart rendering, strategy math, or backtesting.
 - Decouple all analytical calculations from database persistence so offline local dev can run without active PostgreSQL connections.
- Notification Dispatcher Isolation:
 - Verify that the Telegram notification service and interactive bot respect an explicit enablement flag (e.g., TELEGRAM_ENABLED=false in local dev).
 - Ensure local testing never fires alerts to the production Telegram channel or attempts to bind to the production bot polling session.
- Stealth & Decoy Route Structuring:
 - Establish clean architectural routing separating the root public entry point from the internal authenticated trading cockpit and agent routes.
 - Ensure unauthenticated public requests to the root apex return a generic, non-financial telemetry/health response while internal dashboards remain gated behind auth mi

Audit the codebase against these requirements, identify any hardcoded dependencies or cross-environment bleed risks, and report the hardening plan.